Resumo para a Prova A2

Desenvolvimento de Software Visual. Ciência da Computação, 4º semestre, 2026/2. Prof. Diogo Steinke Deconto. Prova A2: **28/09/2026**.

O que foi visto até a prova, pelo calendário de aulas:

Data	Aula	Neste resumo
10/08	Apresentação, configuração de ambiente, ambientação ao C#	§1, §8
17/08	C#: orientação a objetos, classe, atributos, getters/setters, objetos	§2
24/08	Minimal APIs: funcionamento da web, criação do projeto, GET e POST	§3, §4
31/08 e 01/09	Minimal APIs: GET, POST, PUT, PATCH e DELETE	§4, §5
14 e 21/09	Minimal APIs: persistência com Entity Framework	§6

1. Fundamentos de C#
2. Orientação a Objetos
3. Como a Web funciona: HTTP, REST, JSON
4. Minimal APIs no ASP.NET Core
5. Código da aula comentado + CRUD
6. Entity Framework Core
7. Lista de exercícios (Bubble Sort)
8. Comandos da CLI (dotnet)
9. Pegadinhas de prova
10. Questões de revisão com gabarito
11. Folha de cola final

1. Fundamentos de C#

1.1 Estrutura de um programa

Desde o .NET 6 existem os **top-level statements**: o `Program.cs` pode ter o código direto, sem `class Program { static void Main() }`, porque o compilador gera isso. Com `<ImplicitUsings>enable</ImplicitUsings>` no `.csproj`, namespaces como `System`, `System.Linq` e `System.Collections.Generic` já vêm importados.

```
// Program.cs de um projeto console
Console.Clear();
Console.WriteLine("Olá, mundo!");   // imprime e pula linha
Console.Write("sem pular ");         // imprime sem pular linha
string? linha = Console.ReadLine(); // lê do teclado (sempre string, pode ser null)
```

1.2 Tipos de dados

Tipo	Exemplo	Observação
`int`	`int idade = 20;`	inteiro de 32 bits
`long`	`long x = 10L;`	inteiro de 64 bits
`double`	`double h = 1.75;`	decimal padrão (literal sem sufixo)
`float`	`float p = 70.5f;`	precisa do sufixo `f`
`decimal`	`decimal preco = 19.90m;`	sufixo `m`, ideal para dinheiro
`bool`	`bool ativo = true;`	`true` / `false`
`char`	`char c = 'A';`	aspas **simples**
`string`	`string n = "Ana";`	aspas **duplas**, tipo por referência
`var`	`var cidade = "Curitiba";`	tipo inferido na compilação (continua tipado!)
`const`	`const double PI = 3.14;`	valor fixo

Nullable: `string?` e `Produto?` dizem que a variável pode ser `null`. Com `<Nullable>enable</Nullable>` o compilador avisa quando você usa algo que pode ser nulo sem checar. Por isso a API faz `if (produto is null)`.

1.3 Conversões e strings

```
int n = int.Parse("42");                 // lança FormatException se inválido
bool ok = int.TryParse("abc", out int r); // ok = false, r = 0 (não lança)
double d = Convert.ToDouble("2.5");
double m = (double)17 / 5;               // cast: 3.4 (sem cast: 17/5 = 3)
string s = $"Nome: {nome}, preço: {preco:F2}"; // interpolação
```

1.4 Operadores

Aritméticos `+ - * / %`. **Atenção:** `/` entre dois `int` é divisão **inteira** (`17/5 = 3`). `%` é o resto (`17%5 = 2`). Relacionais `== != > < >= <=`. Lógicos `&&` (E), `||` (OU), `!` (NÃO). Incremento `i++`, `+=`. Ternário: `cond ? valorSeVerdadeiro : valorSeFalso`.

1.5 Decisão e repetição

```
if (nota >= 7) { ... }
else if (nota >= 4) { ... }
else { ... }

switch (dia) {
  case 1: ...; break;
  default: ...; break;
}
```

```
for (int i = 0; i < 10; i++) { }  // nº conhecido
while (cond) { }          // testa ANTES
do { } while (cond);      // executa 1x no mínimo
foreach (var x in lista) { } // percorre tudo
// break: sai do laço | continue: pula p/ próxima volta
```

1.6 Vetor (array) × Lista (List<T>)

	Vetor `int[]`	**Lista `List<int>`**
Tamanho	**Fixo**, definido na criação	**Dinâmico**, cresce e diminui
Criar	`new int[100]` ou `{1, 2, 3}`	`new List<int>()` ou `new()`
Tamanho atual	`vetor.Length`	`lista.Count`
Adicionar / remover	não dá	`Add(x)`, `Remove(x)`, `RemoveAt(i)`
Outros	`Array.Sort(v)`, `Array.Reverse(v)`	`Contains`, `IndexOf`, `Clear`, `Sort`

Índices vão de `0` a `Length-1`. Acessar fora disso gera `IndexOutOfRangeException` no vetor e `ArgumentOutOfRangeException` na lista. Um `new int[5]` começa com zeros. `Random r = new Random(); r.Next(1000)` sorteia de **0 a 999**. `r.Next(1, 7)` sorteia de 1 a 6.

1.7 Métodos

```
static int Somar(int a, int b) { return a + b; }  // retorno int
static void Saudar(string n) => Console.WriteLine($"Oi {n}"); // void = não retorna
```

Assinatura = nome + parâmetros. **Sobrecarga** (overload) = mesmo nome com parâmetros diferentes.

1.8 LINQ e expressões lambda

Uma **lambda** é uma função anônima: `p => p.Preco > 100` quer dizer "recebe `p` e devolve se o preço é maior que 100". O **LINQ** (Language Integrated Query) consulta coleções com essas lambdas. No Entity Framework, a mesma consulta é traduzida para **SQL**.

Método	**Faz**	**Equivale em SQL**
`Where(p => ...)`	filtra	`WHERE`
`OrderBy / OrderByDescending`	ordena	`ORDER BY`
`Select(p => p.Nome)`	projeta/transforma	`SELECT Nome`
`FirstOrDefault(p => ...)`	1º que satisfaz, ou `null`/padrão	`... LIMIT 1`
`Any(p => ...)`	existe algum? (`bool`)	`EXISTS`
`Count()` / `Sum()` / `Max()` / `Min()` / `Average()`	agregações	`COUNT/SUM/...`
`ToList()`	executa a consulta e materializa a lista	(executa)

1.9 Exceções

```
try { int x = 10 / zero; }
catch (DivideByZeroException e) { Console.WriteLine(e.Message); }
finally { /* sempre executa */ }
throw new ArgumentException("mensagem");   // lançar uma exceção
```

2. Orientação a Objetos (POO)

2.1 Conceitos

Termo	Definição curta
Classe	Molde, tipo. Define atributos (dados) e métodos (comportamentos). Ex.: `class Produto`.
Objeto	Instância concreta de uma classe, criada com `new`. Ex.: `new Produto()`.
Atributo / campo	Variável que pertence ao objeto e guarda o estado dele.
Propriedade	Jeito C# de expor um atributo com `get` / `set`.
Método	Função da classe, o que o objeto sabe fazer.
Construtor	Método especial, mesmo nome da classe, sem retorno. Roda no `new`.
Encapsulamento	Esconder os dados (`private`) e dar acesso controlado (get/set com validação).
Herança	Classe filha reaproveita a mãe: `class Aluno : Pessoa` ("é um").
Polimorfismo	Mesmo método, comportamento diferente conforme o tipo real (`virtual` / `override`).
Abstração	Modelar só o que importa. Classes `abstract` e interfaces.
Interface	Contrato: diz *o que* a classe tem, não *como*. `interface IAvaliador`.

2.2 Getters e setters: do jeito "Java" à propriedade C#

```
public class Pessoa
{
    // 1) Campo privado + métodos Get/Set (estilo Java)
    private string _cpf = string.Empty;
    public string GetCpf() { return _cpf; }
    public void SetCpf(string cpf)
    {
        if (cpf.Length != 11) throw new ArgumentException("CPF inválido");
        _cpf = cpf;
    }

    // 2) Propriedade AUTOMÁTICA: o compilador cria o campo escondido
    public string Nome { get; set; } = string.Empty;

    // 3) Propriedade COMPLETA, com validação no set ("value" = valor atribuído)
    private int _idade;
    public int Idade
    {
        get { return _idade; }
        set { if (value < 0) throw new ArgumentException("negativa"); _idade = value; }
    }

    // 4) Somente leitura, calculada
    public bool MaiorDeIdade => Idade >= 18;

    public Pessoa() { }                                     // construtor vazio
    public Pessoa(string nome, int idade) { Nome = nome; Idade = idade; } // sobrecarga

    public virtual string Apresentar() => $"Sou {Nome}";
}
```

```
// Três formas de criar objetos
Pessoa p1 = new Pessoa();  p1.Nome = "Ana";  p1.Idade = 17;
Pessoa p2 = new Pessoa("Bruno", 30);                    // construtor
Pessoa p3 = new Pessoa { Nome = "Carla", Idade = 25 };   // inicializador de objeto
```

2.3 Modificadores de acesso

Modificador	Quem acessa
`public`	qualquer código
`private`	só a própria classe (**padrão** dos membros de classe)
`protected`	a própria classe e as filhas
`internal`	o mesmo projeto/assembly (**padrão** das classes)

2.4 Herança, polimorfismo e interface

```
public class Aluno : Pessoa                                    // herança
{
    public string Matricula { get; set; } = "";
    public Aluno(string n, int i, string m) : base(n, i) { Matricula = m; } // chama construtor da
mãe
    public override string Apresentar() => base.Apresentar() + $" ({Matricula})";
}
public interface IAvaliador { double Avaliar(Aluno a); }
public class Professor : Pessoa, IAvaliador { ... }           // 1 classe mãe + N interfaces

List<Pessoa> pessoas = new() { new Pessoa(...), new Aluno(...) };
foreach (var p in pessoas) Console.WriteLine(p.Apresentar()); // polimorfismo
```

> **Lembre**
> C# tem herança **simples** de classes (só uma mãe) e pode implementar **várias interfaces**. Só dá para usar `override` em método marcado `virtual` ou `abstract` na mãe. Toda classe herda de `object` (por isso existe `ToString()`).

2.5 Relacionamento entre classes

- **Associação / composição ("tem um"):** `public Professor? Professor { get; set; }`
- **1 para N ("tem vários"):** `public List<Aluno> Alunos { get; set; } = new();`
- **Herança ("é um"):** `Aluno : Pessoa`

```
public class Turma
{
    public Professor? Professor { get; set; }
    public List<Aluno> Alunos { get; set; } = new();
    public void Matricular(Aluno a) { if (!Alunos.Any(x => x.Matricula == a.Matricula))
Alunos.Add(a); }
}
```

> **Referência × valor**
> Classes são **tipos por referência**: `Pessoa b = a;` copia só o endereço, então mudar `b.Nome` muda `a.Nome`. `int`, `double`, `bool`, `char` e `struct` são **por valor** (a atribuição copia o valor).

3. Como a Web funciona: HTTP, REST e JSON

3.1 Cliente-servidor

O **cliente** (navegador, app, Postman, REST Client) envia uma **requisição**. O **servidor** (a nossa API) processa e devolve uma **resposta**. O HTTP é **sem estado** (stateless): cada requisição é independente.

Requisição tem

- **Método HTTP** (GET, POST...)
- **URL** (endereço/rota)
- Cabeçalhos (ex.: `Content-Type: application/json`)
- Opcional: **corpo** (JSON) ou **parâmetros na URL**

Resposta tem

- **Código de status HTTP** (200, 404...)
- Cabeçalhos
- Corpo: o dado, informação ou mensagem

Essa divisão aparece nos comentários do `Program.cs` da aula: "Requisições: método HTTP, URL, opcional corpo/parâmetros de URL. Resposta: dado/informação/mensagem, código de status HTTP".

3.2 Anatomia de uma URL

```
http://localhost:5195/api/produto/buscar/Mouse?ordem=preco
└──┘   └────────┘ └──┘ └──────────────────────┘ └──────────┘
protocolo host  porta      rota (path)        query string
                              └ "Mouse" = parâmetro de ROTA {nome}
```

3.3 Métodos HTTP (verbos) e CRUD

Método	CRUD	Uso	Tem corpo?	Idempotente?	Status típico
GET	Read	buscar/listar	não	sim	200 OK, 404
POST	Create	cadastrar	sim	**não**	201 Created, 400
PUT	Update	alterar **o recurso inteiro**	sim	sim	200 OK, 404
PATCH	Update	alterar **parte** do recurso	sim (ou query)	não necessariamente	200 OK, 404
DELETE	Delete	remover	normalmente não	sim	200 / 204, 404

Idempotente = repetir a mesma requisição várias vezes deixa o servidor no mesmo estado que uma vez só. Dez POSTs criam dez produtos. Dez DELETEs do mesmo id deixam o produto removido (os seguintes só devolvem 404).

3.4 Códigos de status

Faixa	Significado	Principais	No C# (`Results.`)
1xx	Informativo	100 Continue	—
2xx	**Sucesso**	200 OK · 201 Created · 204 No Content	`Ok()` · `Created()` · `NoContent()`
3xx	Redirecionamento	301 · 302 · 304	`Redirect()`
4xx	**Erro do cliente**	400 Bad Request · 401 Unauthorized · 403 Forbidden · 404 Not Found · 409 Conflict	`BadRequest()` · `Unauthorized()` · `Forbid()` · `NotFound()` · `Conflict()`
5xx	**Erro do servidor**	500 Internal Server Error · 503	`Problem()` / exceção não tratada

3.5 REST e API

- **API** (Application Programming Interface): um conjunto de endpoints que outro programa (o front-end em React, por exemplo) consome.
- **Web API**: API acessada por HTTP.
- **REST** (Representational State Transfer): estilo de arquitetura em que tudo é **recurso** identificado por URL (`/api/produto`), manipulado pelos **verbos HTTP**, com comunicação **sem estado** e representação geralmente em **JSON**. Uma API que segue isso é chamada de **RESTful**.
- **Endpoint**: a combinação **método + rota** que executa uma funcionalidade. `GET /api/produto/listar` e `POST /api/produto/cadastrar` são endpoints diferentes.

3.6 JSON

JSON (JavaScript Object Notation) é um formato de texto para trocar dados. Tem objetos `{ }` com pares `"chave": valor`, listas `[ ]`, strings entre aspas duplas, números, `true`/`false` e `null`.

```
{
  "id": "1d27...bd19",
  "nome": "Mouse",
  "preco": 50,
  "quantidade": 10,
  "criadoEm": "2026-09-28T13:02:41"
}
```

Serialização: objeto C# → JSON (na resposta). **Desserialização:** JSON → objeto C# (o corpo do POST vira um `Produto`).

O ASP.NET faz isso sozinho (System.Text.Json). As propriedades em **PascalCase** do C# (`Nome`) viram **camelCase** no JSON (`nome`), e a leitura não diferencia maiúsculas de minúsculas.

4. Minimal APIs no ASP.NET Core

4.1 O que é

ASP.NET Core é o framework web do .NET. As **Minimal APIs** declaram endpoints direto no `Program.cs` com `app.MapGet/MapPost/...` e uma lambda, sem precisar de *Controllers* (o modelo MVC/Web API "tradicional"). O projeto é criado com `dotnet new web`.

4.2 Esqueleto

```
var builder = WebApplication.CreateBuilder(args); // 1. configura: serviços, DI, config
// builder.Services.AddDbContext<AppDataContext>();  (registrar serviços AQUI)
var app = builder.Build();                           // 2. constrói a aplicação

app.MapGet("/", () => "API do Ecommerce");          // 3. mapeia endpoints

app.Run();                                           // 4. sobe o servidor (bloqueia; o que vier depois não roda)
```

4.3 Mapeando os verbos

Método	Exemplo
`app.MapGet`	`app.MapGet("/api/produto/listar", () => ...)`
`app.MapPost`	`app.MapPost("/api/produto/cadastrar", (Produto produto) => ...)`
`app.MapPut`	`app.MapPut("/api/produto/alterar/{id}", (string id, Produto p) => ...)`
`app.MapPatch`	`app.MapPatch("/api/produto/estoque/{id}", (string id, int quantidade) => ...)`
`app.MapDelete`	`app.MapDelete("/api/produto/remover/{id}", (string id) => ...)`

4.4 De onde vêm os parâmetros (binding)

Origem	Atributo explícito	Exemplo	Regra quando não se usa o atributo
Rota	`[FromRoute]`	`/buscar/{nome}` + `(string nome)`	o nome do parâmetro bate com `{...}` na rota
Query string	`[FromQuery]`	`/estoque/{id}?quantidade=5`	tipo simples que **não** está na rota
Corpo (JSON)	`[FromBody]`	`(Produto produto)`	tipo complexo (classe), em POST/PUT/PATCH
Serviço (DI)	`[FromServices]`	`(AppDataContext ctx)`	tipo registrado em `builder.Services`

`[FromRoute]`, `[FromQuery]`, `[FromBody]` e `[FromServices]` vêm de `using Microsoft.AspNetCore.Mvc;`.

4.5 Retornos: a classe `Results`

O endpoint devolve um `IResult` com status e corpo: `Results.Ok(obj)` → 200, `Results.Created(url, obj)` → 201, `Results.NoContent()` → 204, `Results.BadRequest(msg)` → 400, `Results.NotFound(msg)` → 404, `Results.Conflict(msg)` → 409. Se a lambda devolve uma `string` simples, sai 200 com texto puro. Se devolve um objeto, sai 200 com JSON.

> **Pegadinha**
> Quando a lambda tem vários `return`, todos precisam devolver `Results.X(...)` (o mesmo tipo `IResult`). Misturar `return produtos;` com `return Results.NotFound()` não compila.

4.6 Rodando e testando

- `dotnet run` (ou `dotnet watch run`, que reinicia sozinho quando o código muda). A porta vem de `Properties/launchSettings.json` (`"applicationUrl": "http://localhost:5195"`). `appsettings.json` guarda configurações como logging e connection strings.
- Para testar: navegador (só GET), **Postman**, **Insomnia**, a extensão **REST Client** do VS Code com arquivo `.http`, ou `curl`.

```
### Arquivo Tests/Produto.http (REST Client). "###" separa as requisições
GET http://localhost:5195/api/produto/listar

###
POST http://localhost:5195/api/produto/cadastrar
Content-Type: application/json
                                  <- linha em branco OBRIGATÓRIA antes do corpo
{ "nome": "Teclado", "preco": 150, "quantidade": 5 }
```

5. Código da aula comentado + CRUD completo

5.1 Modelo

```
public class Produto
{
    public string Id { get; set; } = Guid.NewGuid().ToString(); // id único gerado ao criar o
objeto
    public string Nome { get; set; } = String.Empty;
    public DateTime CriadoEm { get; set; } = DateTime.Now;     // data/hora da criação
}
```

GUID (Globally Unique Identifier) é um identificador de 128 bits, praticamente impossível de repetir, por exemplo `1d274707-7f49-45f1-a066-f34c3697bd19`. Os valores depois do `=` são **inicializadores de propriedade**: rodam em todo `new Produto()`, inclusive quando o JSON é desserializado.

5.2 Endpoints da aula (GET e POST), explicados

```
List<Produto> produtos = new List<Produto>();   // "banco" EM MEMÓRIA: some quando a API para

app.MapGet("/api/produto/listar", () =>
{
    if (produtos.Count == 0) return Results.BadRequest("A lista de produtos está vazia");
    return Results.Ok(produtos);                    // 200 + JSON da lista
});

app.MapPost("/api/produto/cadastrar", (Produto? produto) =>   // vem do CORPO (JSON)
{
    if (produto is null) return Results.BadRequest("O produto não pode ser nulo");
    if (produto.Nome == "") return Results.BadRequest("O nome não pode ser vazio");
    foreach (Produto cadastrado in produtos)        // regra: nome único
        if (cadastrado.Nome == produto.Nome) return Results.BadRequest("Já existe um produto com o
mesmo nome");
    produtos.Add(produto);
    return Results.Created("", produto);           // 201 Created
});

app.MapGet("/api/produto/buscar/{nome}", (string nome) =>   // vem da ROTA
{
    foreach (Produto cadastrado in produtos)
        if (cadastrado.Nome == nome) return Results.Ok(cadastrado);
    return Results.NotFound("Produto não encontrado!"); // 404
});
```

5.3 Exercício da aula resolvido: remoção e alteração

```
//DELETE: /api/produto/remover/{id}
app.MapDelete("/api/produto/remover/{id}", (string id) =>
{
    Produto? resultado = produtos.FirstOrDefault(p => p.Id == id);  // LINQ no lugar do foreach
    if (resultado is null) return Results.NotFound("Produto não encontrado!");
    produtos.Remove(resultado);
    return Results.Ok(resultado);
});

//PUT: /api/produto/alterar/{id}  (id na ROTA + dados novos no CORPO)
app.MapPut("/api/produto/alterar/{id}", (string id, Produto produtoAlterado) =>
{
    Produto? resultado = produtos.FirstOrDefault(p => p.Id == id);
    if (resultado is null) return Results.NotFound("Produto não encontrado!");
    resultado.Nome = produtoAlterado.Nome;          // Id e CriadoEm continuam os mesmos
    resultado.Preco = produtoAlterado.Preco;
    resultado.Quantidade = produtoAlterado.Quantidade;
    return Results.Ok(resultado);
});

//PATCH: /api/produto/estoque/{id}?quantidade=20  (altera SÓ um campo)
app.MapPatch("/api/produto/estoque/{id}", (string id, int quantidade) =>
{
    Produto? resultado = produtos.FirstOrDefault(p => p.Id == id);
    if (resultado is null) return Results.NotFound("Produto não encontrado!");
    resultado.Quantidade = quantidade;
    return Results.Ok(resultado);
});
```

> **Por que `FirstOrDefault` funciona para alterar?**
> Ele devolve a **referência** do objeto que está dentro da lista. Alterar `resultado.Nome` altera o objeto guardado na lista (tipo por referência, §2.5). Por isso não é preciso "salvar de volta".

Por que usar o `Id` e não o nome para alterar ou remover? Porque o nome pode mudar (é justamente o que o PUT altera) e o Id é o identificador estável e único do recurso.

6. Entity Framework Core (persistência)

6.1 Conceitos

- **Persistência:** guardar os dados de forma que sobrevivam ao fim do programa. A `List` em memória some quando a API para. O banco de dados não.
- **ORM** (Object-Relational Mapping, mapeamento objeto-relacional): converte automaticamente entre **objetos** C# e **tabelas/linhas** do banco relacional. Classe → tabela. Propriedade → coluna. Objeto → linha. Você escreve LINQ e o ORM gera o SQL.
- **Entity Framework Core (EF Core)**: o ORM oficial do .NET. Funciona com vários bancos por meio de **providers** (SQLite, SQL Server, MySQL, PostgreSQL).
- **SQLite:** banco que fica num **arquivo** local (`Ecommerce.db`), sem servidor para instalar. Bom para estudo.
- **Code First:** você escreve as classes primeiro e o EF gera o banco a partir delas (via **migrations**).

6.2 Passo a passo

1. Instalar os pacotes NuGet no projeto:

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite    # provider do banco
dotnet add package Microsoft.EntityFrameworkCore.Design    # necessário para migrations
dotnet tool install --global dotnet-ef                     # ferramenta de linha de comando
(uma vez)
```

2. Criar o **contexto** (herda de `DbContext`):

```
using Microsoft.EntityFrameworkCore;
public class AppDataContext : DbContext
{
    public DbSet<Produto> Produtos { get; set; }     // cada DbSet = uma TABELA
    public DbSet<Categoria> Categorias { get; set; }
    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseSqlite("Data Source=Ecommerce.db");  // connection string
    }
}
```

3. Registrar no `Program.cs` (**injeção de dependência**):
   `builder.Services.AddDbContext<AppDataContext>();` (antes do `builder.Build()`).
4. Gerar e aplicar a migration (dentro da pasta do projeto):

```
dotnet ef migrations add InicialCriacaoTabelas   # gera a pasta Migrations/ com o código C# que
cria as tabelas
dotnet ef database update                        # executa no banco: cria o Ecommerce.db e as
tabelas
```

5. Receber o contexto no endpoint: `([FromServices] AppDataContext ctx) => ...`

> **Migration**
> É o "controle de versão" do esquema do banco. Sempre que mudar uma classe (nova propriedade, nova entidade), rode `migrations add NovoNome` e depois `database update`. A tabela `__EFMigrationsHistory` registra quais migrations já foram aplicadas. `dotnet ef migrations remove` desfaz a última que ainda não foi aplicada.

6.3 CRUD com o contexto

Operação	Código EF Core	SQL gerado
Listar	`ctx.Produtos.ToList()`	`SELECT * FROM Produtos`
Buscar pela PK	`ctx.Produtos.Find(id)`	`SELECT ... WHERE Id = @id`
Buscar com filtro	`ctx.Produtos.FirstOrDefault(p => p.Nome == nome)`	`... WHERE Nome = @nome LIMIT 1`
Filtrar/ordenar	`ctx.Produtos.Where(p => p.Preco <= max).OrderBy(p => p.Preco).ToList()`	`WHERE ... ORDER BY ...`
Inserir	`ctx.Produtos.Add(p); ctx.SaveChanges();`	`INSERT`
Alterar	`ctx.Produtos.Update(p); ctx.SaveChanges();`	`UPDATE`
Remover	`ctx.Produtos.Remove(p); ctx.SaveChanges();`	`DELETE`
Trazer relacionado	`ctx.Produtos.Include(p => p.Categoria).ToList()`	`LEFT JOIN Categorias`

> **SaveChanges()**
> `Add`, `Update` e `Remove` só **marcam** a mudança no *change tracker*. Nada vai para o banco até chamar `ctx.SaveChanges()`. Esquecer o `SaveChanges` é o erro nº 1. Um objeto obtido por `Find` já é rastreado: basta alterar as propriedades e chamar `SaveChanges()`.

6.4 Endpoint completo com EF

```
app.MapPost("/api/produto/cadastrar", ([FromBody] Produto produto, [FromServices] AppDataContext
ctx) =>
{
    if (ctx.Produtos.Any(p => p.Nome == produto.Nome))
        return Results.Conflict("Já existe um produto com o mesmo nome");
    ctx.Produtos.Add(produto);
    ctx.SaveChanges();
    return Results.Created($"/api/produto/buscar/{produto.Id}", produto);
});

app.MapDelete("/api/produto/remover/{id}", ([FromRoute] string id, [FromServices] AppDataContext
ctx) =>
{
    Produto? produto = ctx.Produtos.Find(id);
    if (produto is null) return Results.NotFound("Produto não encontrado!");
    ctx.Produtos.Remove(produto);
    ctx.SaveChanges();
    return Results.Ok(produto);
});
```

6.5 Convenções e Data Annotations

- Uma propriedade chamada `Id` ou `<Classe>Id` vira a **chave primária** automaticamente (ou use `[Key]`).
- **Relacionamento 1:N**: `public string? CategoriaId { get; set; }` (chave estrangeira) + `public Categoria? Categoria { get; set; }` (propriedade de navegação).
- Tipos anuláveis (`string?`) viram colunas que aceitam NULL.

```
using System.ComponentModel.DataAnnotations;
public class Produto
{
    [Key] public string Id { get; set; } = Guid.NewGuid().ToString();
    [Required(ErrorMessage = "O nome é obrigatório")]
    [MinLength(3)] [StringLength(100)]         // StringLength/MaxLength também limitam a coluna
    public string Nome { get; set; } = "";
    [Range(0.01, 100000)] public double Preco { get; set; }
    [EmailAddress] public string? EmailFornecedor { get; set; }
}
```

Outras annotations: `[MaxLength]`, `[RegularExpression]`, `[Compare]`, `[Phone]`, `[NotMapped]` (não vira coluna), `[Table("Nome")]`, `[Column("nome")]`.

> **Minimal API não valida sozinha (.NET 8)**
>
> Nos *Controllers* com `[ApiController]` a validação é automática. Na Minimal API do .NET 8 é preciso validar manualmente:
>
> ```
> List<ValidationResult> erros = new();
> if (!Validator.TryValidateObject(produto, new ValidationContext(produto), erros, true))
>     return Results.BadRequest(erros);   // 400 com a lista de mensagens
> ```

6.6 Injeção de dependência (DI)

Em vez de o endpoint criar o próprio `new AppDataContext()`, ele **recebe** um pronto do framework. O `AddDbContext` registra o contexto com tempo de vida **Scoped**: uma instância por requisição HTTP, descartada no final. Isso desacopla o código e facilita testes. Tempos de vida: **Singleton** (uma instância para sempre), **Scoped** (uma por requisição), **Transient** (uma nova a cada pedido).

7. Lista de exercícios: ordenação (Exe06)

Enunciado: gerar um vetor com 100 números aleatórios e ordená-lo, (a) manualmente e (b) com recurso da linguagem.

```
int[] vetor = new int[100];
Random random = new Random();
for (int i = 0; i < vetor.Length; i++) vetor[i] = random.Next(1000);

// (a) BUBBLE SORT: compara vizinhos e troca se estiverem fora de ordem
bool troca;
do
{
    troca = false;
    for (int i = 0; i < vetor.Length - 1; i++)   // -1 porque acessa vetor[i + 1]
    {
        if (vetor[i] > vetor[i + 1])
        {
            troca = true;
            int aux = vetor[i];                    // troca com variável auxiliar
            vetor[i] = vetor[i + 1];
            vetor[i + 1] = aux;
        }
    }
} while (troca);                                  // repete até uma passada sem trocas

// (b) Exe06b: recurso pronto
Array.Sort(vetor);
```

> **Erro comum (estava na versão do zip)**
> Só um `for`, sem o `do-while`: uma passada leva o **maior** elemento para o final, mas o vetor **não** fica ordenado. É preciso repetir as passadas (no pior caso, n−1 vezes). Complexidade do Bubble Sort: **O(n²)**. `Array.Sort`: **O(n log n)**.

Teste de mesa de uma passada em `[5, 1, 4, 2]`: troca 5↔1 → `[1,5,4,2]`. Troca 5↔4 → `[1,4,5,2]`. Troca 5↔2 → `[1,4,2,5]` (houve troca, então repete). Segunda passada → `[1,2,4,5]`. Terceira passada: nenhuma troca, fim.

8. Comandos da CLI (dotnet)

Comando	O que faz
`dotnet --version`	versão do SDK instalada
`dotnet new sln -n Ecommerce`	cria um arquivo de **solução** (.sln), que agrupa projetos
`dotnet new console -o Exe06a`	cria projeto de **console**
`dotnet new web -o API`	cria projeto **web vazio** (Minimal API)
`dotnet new webapi -o API`	cria Web API com exemplo (WeatherForecast) e Swagger
`dotnet sln add API`	vincula o projeto à solução
`dotnet build`	compila (mostra erros e warnings)
`dotnet run` / `dotnet watch run`	compila e executa / executa reiniciando a cada alteração
`dotnet add package <Nome>`	instala pacote do **NuGet** (gerenciador de pacotes do .NET)
`dotnet restore`	baixa os pacotes listados no .csproj
`dotnet tool install --global dotnet-ef`	instala a ferramenta do EF
`dotnet ef migrations add Nome`	cria uma migration
`dotnet ef database update`	aplica as migrations no banco

Arquivos importantes: `.sln` (solução), `.csproj` (projeto: framework-alvo `net8.0`, pacotes), `Program.cs` (ponto de entrada), `Properties/launchSettings.json` (portas e perfis), `appsettings.json` (configurações), pastas `bin/` e `obj/` (saída da compilação, ficam no `.gitignore`).

9. Pegadinhas de prova

1. `17 / 5` dá **3** (divisão inteira). Use `17 / 5.0` ou um cast.
2. `=` é atribuição. `==` é comparação.
3. Vetor usa `.Length`. Lista usa `.Count`. String usa `.Length`.
4. `random.Next(1000)` vai até **999**. O limite superior é exclusivo.
5. `for (i = 0; i <= v.Length; i++)` estoura o vetor. O certo é `<`.
6. O construtor não tem tipo de retorno (nem `void`) e tem o mesmo nome da classe.
7. Sem `virtual` na mãe, não há `override`.
8. Criar um objeto de uma classe `abstract` ou de uma interface com `new` não é permitido.
9. POST **não** é idempotente. PUT substitui tudo. PATCH altera parte.
10. 404 = recurso não encontrado (erro do cliente). 500 = erro no servidor. 400 = requisição inválida. 201 = criado.
11. No `.http` falta a linha em branco antes do corpo, ou o `Content-Type: application/json`, e o corpo chega nulo.
12. Código depois de `app.Run()` não executa (enquanto a API estiver rodando). Serviços são registrados **antes** do `builder.Build()`.
13. Esquecer `ctx.SaveChanges()`: nada é gravado.
14. Mudou o modelo e não rodou `migrations add` + `database update`: erro "no such column/table".
15. `Find` só busca pela **chave primária**. Para outros campos, use `FirstOrDefault`/`Where`.
16. `FirstOrDefault` devolve `null` quando não acha, então sempre cheque `is null` antes de usar.
17. LINQ tem execução **adiada**: a consulta só roda no `ToList()`, `First()`, `Count()` etc.
18. Os dados em `List` na memória somem quando a API reinicia. Por isso existe o EF/banco.

10. Questões de revisão (com gabarito)

Objetivas

1. Qual método HTTP é o mais indicado para **cadastrar** um novo produto?
a) GET b) POST c) PUT d) DELETE

> b) POST. Cria um recurso novo e responde 201 Created.

2. Uma API responde **404**. Isso significa:
a) sucesso sem conteúdo b) erro interno do servidor c) recurso não encontrado d) requisição criada

> c) 404 Not Found (erro do cliente, faixa 4xx).

3. Qual a diferença entre PUT e PATCH?

> PUT substitui o recurso inteiro (manda todos os campos). PATCH altera parcialmente (só os campos enviados).

4. Em `app.MapGet("/api/produto/buscar/{nome}", (string nome) => ...)`, de onde vem `nome`?
a) corpo b) query string c) rota d) cabeçalho

> c) Rota: o segmento `{nome}` da URL.

5. Qual o resultado de `Console.WriteLine(7 / 2 + 7 % 2);`?

> 4. `7/2 = 3` (divisão inteira) e `7%2 = 1`.

6. O que acontece se você chamar `ctx.Produtos.Add(p)` e não chamar `ctx.SaveChanges()`?

O produto só fica marcado no change tracker. Nenhum INSERT é executado e nada é gravado no banco.

7. Qual pilar da POO permite que `pessoa.Apresentar()` execute a versão de `Aluno` quando o objeto real é um aluno?

Polimorfismo (método `virtual` na mãe e `override` na filha).

8. Qual comando cria uma migration?
a) `dotnet ef database update` b) `dotnet ef migrations add Nome` c) `dotnet new migration` d) `dotnet add migration`

b). O a) aplica as migrations no banco.

9. O que é um ORM?

Mapeamento objeto-relacional: uma camada que converte automaticamente objetos/classes em linhas/tabelas de um banco relacional e gera o SQL (ex.: Entity Framework Core).

10. Qual status deve retornar um cadastro bem-sucedido, e qual método de `Results` o produz?

201 Created, com `Results.Created(url, objeto)`.

Discursivas / código

11. Explique encapsulamento e mostre como validar que o preço de um produto não seja negativo.

Encapsulamento é esconder o estado interno (`private`) e expor acesso controlado. Exemplo:

```
private double _preco;
public double Preco {
    get { return _preco; }
    set { if (value < 0) throw new ArgumentException("Preço negativo"); _preco = value; }
}
```

12. Escreva um endpoint que devolva apenas os produtos com quantidade maior que zero (lista em memória `produtos`).

```
app.MapGet("/api/produto/disponiveis", () =>
    Results.Ok(produtos.Where(p => p.Quantidade > 0).ToList()));
```

13. Escreva o endpoint de remoção por id usando Entity Framework.

```
app.MapDelete("/api/produto/remover/{id}", ([FromRoute] string id, [FromServices]
AppDataContext ctx) => {
    Produto? p = ctx.Produtos.Find(id);
    if (p is null) return Results.NotFound("Produto não encontrado!");
    ctx.Produtos.Remove(p);
    ctx.SaveChanges();
    return Results.Ok(p);
});
```

14. Encontre os erros:

```
app.MapPost("/api/produto/cadastrar", (Produto produto, AppDataContext ctx) => {
    ctx.Produtos.Add(produto);
    return produto;
});
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDataContext>();
```

(1) O endpoint é mapeado antes de existir `app`, e o `AddDbContext` tem que vir antes do `builder.Build()`. A ordem certa é builder → services → Build → Map → Run. (2) Falta `ctx.SaveChanges()`. (3) O ideal é devolver `Results.Created(...)` com 201, em vez do objeto (que sai como 200).

15. Monte o arquivo `.http` para alterar o produto de id `abc` com nome "Mouse Gamer" e preço 120.

```
PUT http://localhost:5195/api/produto/alterar/abc
Content-Type: application/json

{ "nome": "Mouse Gamer", "preco": 120, "quantidade": 1 }
```

16. Modele: uma `Categoria` tem vários `Produto`s. Como fica no EF?

Em `Produto`: `public string? CategoriaId { get; set; }` (FK) e `public Categoria? Categoria { get; set; }` (navegação). No contexto: `DbSet<Categoria> Categorias`. Para trazer junto: `ctx.Produtos.Include(p => p.Categoria)`. Depois, `migrations add` + `database update`.

17. Por que a API em memória perde os dados e como o EF resolve?

A `List<Produto>` vive na RAM do processo e some quando a API para. O EF grava num banco (arquivo SQLite, por exemplo), que persiste entre execuções.

11. Folha de cola final

```
// ===== Program.cs de uma Minimal API com EF =====
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;

var builder = WebApplication.CreateBuilder(args);
builder.Services.AddDbContext<AppDataContext>();
var app = builder.Build();

app.MapGet("/api/produto/listar", ([FromServices] AppDataContext ctx) =>
    ctx.Produtos.Any() ? Results.Ok(ctx.Produtos.ToList()) : Results.NotFound("Vazio"));

app.MapGet("/api/produto/buscar/{id}", ([FromRoute] string id, [FromServices] AppDataContext ctx)
=>
    ctx.Produtos.Find(id) is Produto p ? Results.Ok(p) : Results.NotFound());

app.MapPost("/api/produto/cadastrar", ([FromBody] Produto p, [FromServices] AppDataContext ctx) =>
{ ctx.Produtos.Add(p); ctx.SaveChanges(); return Results.Created($"/api/produto/buscar/{p.Id}",
p); });

app.MapPut("/api/produto/alterar/{id}", ([FromRoute] string id, [FromBody] Produto novo,
[FromServices] AppDataContext ctx) =>
{
    Produto? p = ctx.Produtos.Find(id);
    if (p is null) return Results.NotFound();
    p.Nome = novo.Nome; p.Preco = novo.Preco; p.Quantidade = novo.Quantidade;
    ctx.Produtos.Update(p); ctx.SaveChanges(); return Results.Ok(p);
});

app.MapDelete("/api/produto/remover/{id}", ([FromRoute] string id, [FromServices] AppDataContext
ctx) =>
{
    Produto? p = ctx.Produtos.Find(id);
    if (p is null) return Results.NotFound();
    ctx.Produtos.Remove(p); ctx.SaveChanges(); return Results.Ok(p);
});

app.Run();
```

Lembrar	
Criar API	`dotnet new web -o API` → `dotnet run`
EF	pacotes Sqlite + Design → `DbContext` + `DbSet` + `UseSqlite` → `AddDbContext` → `migrations add` → `database update`
Status	200 OK · 201 Created · 204 No Content · 400 Bad Request · 404 Not Found · 409 Conflict · 500 Server Error
Verbos	GET ler · POST criar · PUT substituir · PATCH alterar parte · DELETE remover
Parâmetros	rota `{id}` · query `?x=1` · corpo JSON · serviços (DI)
POO	classe/objeto · encapsulamento · herança `:` · polimorfismo `virtual/override` · interface · abstração

Material complementar de estudo, montado a partir do Plano de Ensino, do calendário de aulas e dos códigos do repositório da disciplina. Os códigos completos, que compilam e foram testados, estão no repositório de estudos que acompanha este PDF.